

Ejercicios de práctica

Fundamentos Python (2)

Listas, tuplas y diccionarios

Objetivo	Practicar la selección y el uso de colecciones en Python a partir de problemas pequeños, verificables y acumulativos.
Temas	Listas, tuplas, diccionarios, recorridos con for, filtrado, acumuladores, listas de diccionarios y estructuras anidadas.
Modalidad	Ejercicios incompletos para resolver individualmente. No se incluyen soluciones en este documento.

Cómo trabajar: complete cada bloque marcado con ___, ejecute el código y compare con el resultado esperado antes de avanzar al siguiente ejercicio.

Regla de trabajo

- Los espacios marcados con ___ deben ser completados.
- Ejecute los ejercicios en orden: cada bloque se apoya en el anterior.
- Complete los espacios sin cambiar el resultado esperado.
- Escriba una respuesta breve cuando se solicite una interpretación.
- Identifique en cada caso cuándo conviene una lista, una tupla o un diccionario.

A. Listas

Crear, recorrer, filtrar y acumular

1 Crear una lista de notas

BÁSICO

Complete la lista con cinco notas y muestre la cantidad de elementos.

```
notas = [__, __, __, __, __]

print(len(notas))
print(notas)
```

Resultado esperado: Se imprime 5 y luego la lista completa.

2 Acceder por índice

BÁSICO

Use índices para imprimir la primera, la tercera y la última temperatura.

```
temperaturas = [18, 20, 19, 21, 22]

print(temperaturas[__])
print(temperaturas[__])
print(temperaturas[__])
```

Resultado esperado: 18, 19 y 22 en líneas separadas.

3 Actualizar un elemento

BÁSICO

Corrija la segunda venta, que fue registrada como 80 pero debía ser 85.

```
ventas = [120, 80, 200, 50]

ventas[__] = __
print(ventas)
```

Resultado esperado: [120, 85, 200, 50]

4 Agregar y extraer datos

BÁSICO

Agregue una nueva nota y extraiga la última nota registrada.

```
notas = [3.5, 4.0, 2.8]

notas.__(4.6)
ultima = notas.__()

print(ultima)
print(notas)
```

Resultado esperado: 4.6 y luego [3.5, 4.0, 2.8]

5 Calcular promedio

BÁSICO

Complete el acumulador para calcular el promedio de una lista.

```
notas = [4.2, 3.8, 5.0, 2.9]
total = 0

for nota in notas:
    total = total + ____

promedio = total / ____ (notas)
print(round(promedio, 2))
```

Resultado esperado: 3.98

6 Contar aprobados

BÁSICO

Cuente cuántas notas son mayores o iguales a 3.0.

```
notas = [4.2, 2.5, 3.0, 1.8, 4.7]
aprobados = 0

for nota in notas:
    if nota ____ 3.0:
        aprobados = aprobados + ____

print(aprobados)
```

Resultado esperado: 3

7 Filtrar valores válidos

BÁSICO

Construya una lista con las temperaturas que estén entre -10 y 50 grados.

```
lecturas = [18, 200, 21, -99, 19, 22]
validas = []

for t in lecturas:
    if t >= ____ and t <= ____:
        validas.append(____)

print(validas)
```

Resultado esperado: [18, 21, 19, 22]

8 Ordenar sin perder el original

BÁSICO

Use sorted() para crear una lista ordenada sin modificar la lista inicial.

```
montos = [120000, 85000, 210000, 50000]

ordenados = ____ (montos)

print(montos)
print(ordenados)
```

Resultado esperado: [120000, 85000, 210000, 50000] y [50000, 85000, 120000, 210000]

9 Comprensión de listas

BÁSICO

Complete la comprensión para obtener los cuadrados de los números pares.

```
numeros = [1, 2, 3, 4, 5, 6]

cuadrados_pares = [n**2 for n in numeros if n ____ 2 == 0]
print(cuadrados_pares)
```

Resultado esperado: [4, 16, 36]

10 Lista anidada

INTERMEDIO

Complete el doble ciclo para sumar todos los elementos de una matriz.

```
matriz = [[1, 2, 3], [4, 5, 6]]
total = 0

for fila in matriz:
    for valor in ____:
        total += ____

print(total)
```

Resultado esperado: 21

Respuesta breve: explique en una frase por qué una matriz puede modelarse como una lista de listas.

B. Tuplas

Datos fijos, desempaquetado y uso como clave

11 Crear una tupla fija

BÁSICO

Defina una tupla con la latitud y la longitud de una sede.

```
ubicacion = (____, ____)\n\nprint(ubicacion)\nprint(type(ubicacion))
```

Resultado esperado: Una tupla de dos valores y el tipo <class 'tuple'>.

12 Desempaquetar una tupla

BÁSICO

Asigne los elementos de la tupla a variables con nombres significativos.

```
registro = ("A01", "Laura", 4.6)\n\n____, ____, ____ = registro\n\nprint(nombre)\nprint(nota)
```

Resultado esperado: Laura y 4.6

13 Tupla de un solo elemento

BÁSICO

Complete la sintaxis correcta de una tupla que contiene un solo código.

```
codigo = ("A01"____)\n\nprint(codigo)\nprint(type(codigo))
```

Resultado esperado: ('A01',) y el tipo <class 'tuple'>.

14 Convertir lista a tupla

BÁSICO

Convierta una lista de columnas a tupla para evitar cambios accidentales.

```
columnas = ["fecha", "monto", "cliente"]

columnas_fijas = ____(columnas)
print(columnas_fijas)
```

Resultado esperado: ('fecha', 'monto', 'cliente')

15 Retorno múltiple

INTERMEDIO

Complete una función que retorne el mínimo y el máximo como una tupla implícita.

```
def resumen(valores):
    menor = min(valores)
    mayor = max(valores)
    return ____, ____

minimo, maximo = resumen([8, 3, 10, 5])
print(minimo, maximo)
```

Resultado esperado: 3 10

16 Reconocer inmutabilidad

INTERMEDIO

Ejecute mentalmente el código y explique qué error se produce.

```
punto = (10, 20)

punto[0] = 99
print(punto)
```

Resultado esperado: Debe producir TypeError porque la tupla no permite asignar nuevos valores por índice.

Respuesta breve: ¿por qué esta restricción puede ser útil en un programa real?

17 Tupla como clave

INTERMEDIO

Use una tupla como clave para almacenar una lectura por coordenada.

```
lecturas = {}  
coordenada = (4.65, -74.05)  
  
lecturas[coordenada] = 18.5  
print(lecturas[coordenada])
```

Resultado esperado: 18.5

18 Combinar listas con zip

BÁSICO

Cree pares estudiante-nota usando zip() y conviértalos a lista.

```
nombres = ["Ana", "Luis", "Marta"]  
notas = [4.2, 3.8, 5.0]  
  
pares = list(zip(nombres, notas))  
print(pares)
```

Resultado esperado: [('Ana', 4.2), ('Luis', 3.8), ('Marta', 5.0)]

C. Diccionarios

Pares clave-valor, acumuladores y registros

19 Crear un diccionario

BÁSICO

Complete las claves necesarias para representar a un estudiante.

```
estudiante = {  
    __: "A01",  
    __: "Laura",  
    __: 4.6  
}  
  
print(estudiante)
```

Resultado esperado: Un diccionario con código, nombre y nota.

20 Acceso seguro con get()

BÁSICO

Use get() para consultar una clave que podría no existir.

```
cliente = {"nombre": "Carlos", "puntaje": 720}  
  
saldo = cliente.__("saldo", 0)  
print(saldo)
```

Resultado esperado: 0

21 Actualizar valores

BÁSICO

Actualice el stock de un producto después de una venta de 3 unidades.

```
producto = {"codigo": "P01", "stock": 8}  
  
producto["stock"] = producto["stock"] - ____  
print(producto["stock"])
```

Resultado esperado: 5

22 Recorrer pares clave-valor

BÁSICO

Complete el recorrido para imprimir cada atributo con su valor.

```
producto = {"codigo": "P01", "precio": 12000, "stock": 8}

for clave, valor in producto.__dict__.items():
    print(clave, valor)
```

Resultado esperado: Se imprime cada clave con su valor correspondiente.

23 Contar por categoría

INTERMEDIO

Use un diccionario acumulador para contar los tipos de transacción.

```
tipos = ["Debito", "Credito", "Debito", "Debito", "Credito"]
conteo = {}

for tipo in tipos:
    conteo[tipo] = conteo.get(tipo, 0) + 1

print(conteo)
```

Resultado esperado: {'Debito': 3, 'Credito': 2}

24 Diccionario anidado

INTERMEDIO

Acceda a la nota de Laura dentro de una estructura anidada.

```
grupo = {
    "A01": {"nombre": "Laura", "nota": 4.6},
    "A02": {"nombre": "Luis", "nota": 3.8}
}

print(grupo["A01"]["nota"])
```

Resultado esperado: 4.6

25 Lista de diccionarios

BÁSICO

Calcule el total de los montos en una lista de transacciones.

```
transacciones = [
    {"id": "T01", "monto": 120000},
    {"id": "T02", "monto": 85000},
    {"id": "T03", "monto": 210000}
]
total = 0

for t in transacciones:
    total += t[___]

print(total)
```

Resultado esperado: 415000

26 Buscar por código

INTERMEDIO

Complete una búsqueda lineal sobre una lista de diccionarios.

```
estudiantes = [
    {"codigo": "A01", "nombre": "Laura"},
    {"codigo": "A02", "nombre": "Luis"}
]
buscado = "A02"
resultado = None

for e in estudiantes:
    if e["codigo"] == ___:
        resultado = e["nombre"]

print(resultado)
```

Resultado esperado: Luis

27 Filtrar diccionarios

INTERMEDIO

Construya una lista con los nombres de los estudiantes aprobados.

```
estudiantes = [
    {"nombre": "Ana", "nota": 4.2},
    {"nombre": "Luis", "nota": 2.8},
    {"nombre": "Marta", "nota": 3.5}
]
aprobados = []

for e in estudiantes:
    if e["nota"] >= __:
        aprobados.append(e[__])

print(aprobados)
```

Resultado esperado: ['Ana', 'Marta']

28 Agrupar por estado

AVANZADO

Agrupe los nombres de los clientes según su nivel de riesgo.

```
clientes = [
    {"nombre": "Ana", "riesgo": "bajo"},
    {"nombre": "Luis", "riesgo": "alto"},
    {"nombre": "Marta", "riesgo": "bajo"}
]
grupo = {}

for c in clientes:
    riesgo = c["riesgo"]
    if riesgo not in grupo:
        grupo[riesgo] = __
    grupo[riesgo].append(c[__])

print(grupo)
```

Resultado esperado: {'bajo': ['Ana', 'Marta'], 'alto': ['Luis']}

29 Inventario académico

INTERMEDIO

Complete el reporte de los productos con stock bajo.

```
inventario = [  
    {"producto": "Marcador", "stock": 4},  
    {"producto": "Cuaderno", "stock": 20},  
    {"producto": "Borrador", "stock": 2}  
]  
bajos = []  
  
for item in inventario:  
    if item["stock"] < ____:  
        bajos.append(item[____])  
  
print(bajos)
```

Resultado esperado: ['Marcador', 'Borrador']

Reto integrador

Combina listas de diccionarios, filtros y acumuladores

30 Mini caso integrador

AVANZADO

Complete el algoritmo que calcula el total de los créditos mayores o iguales a 100000 y registra los clientes correspondientes.

```
transacciones = [
    {"cliente": "Ana", "tipo": "Credito", "monto": 120000},
    {"cliente": "Luis", "tipo": "Debito", "monto": 85000},
    {"cliente": "Marta", "tipo": "Credito", "monto": 210000}
]
total_creditos = 0
clientes = []

for t in transacciones:
    if t["tipo"] == ____ and t["monto"] >= ____:
        total_creditos += t["monto"]
        clientes.append(t[____])

print(total_creditos)
print(clientes)
```

Resultado esperado: 330000 y ['Ana', 'Marta']

Respuesta breve: indique qué estructuras aparecen en este caso y para qué se usa cada una.

Cierre y pauta de revisión

Verificación final antes de entregar el cuaderno

Antes de entregar o compartir el cuaderno, verifique que el código se ejecuta de inicio a fin y que los resultados coinciden con los resultados esperados. Cuando un resultado no coincida, revise primero los índices, los nombres de las claves y las condiciones de los ciclos.

Estructura	Uso esperado en los ejercicios
Lista	Almacenar múltiples valores, recorrer, filtrar, ordenar y acumular.
Tupla	Representar datos fijos, desempaquetar y usar como clave cuando corresponde.
Diccionario	Representar atributos con nombre, consultar con <code>get()</code> , actualizar y recorrer con <code>items()</code> .
Lista de diccionarios	Modelar registros de un sistema real y procesarlos con ciclos.

Referencias breves

- Python Software Foundation. The Python Tutorial: Data Structures.
- Joyanes Aguilar (2020). Fundamentos de programación.
- Ramírez Jiménez (2021). Python a Fondo.

Autoevaluación rápida

1. ¿Puedo elegir entre lista, tupla y diccionario según el problema?
2. ¿Puedo recorrer una colección y acumular un resultado?
3. ¿Puedo filtrar registros dentro de una lista de diccionarios?
4. ¿Puedo explicar por qué una tupla no admite asignación por índice?